

13

Collision and Rigid Body Dynamics

In the real world, solid objects are inherently, well...solid. They generally avoid doing impossible things, like passing through one another, all by themselves. But in a virtual game world, objects don't do anything unless we tell them to, and game programmers must make an explicit effort to ensure that objects do not pass through one another. This is the role of one of the central components of any game engine—the *collision detection system*.

A game engine's collision system is often closely integrated with a *physics engine*. Of course, the field of physics is vast, and what most of today's game engines call "physics" is more accurately described as a *rigid body dynamics* simulation. A *rigid body* is an idealized, infinitely hard, non-deformable solid object. The term *dynamics* refers to the process of determining how these rigid bodies *move* and *interact* over time under the influence of *forces*. A rigid body dynamics simulation allows motion to be imparted to objects in the game in a highly interactive and naturally chaotic manner—an effect that is much more difficult to achieve when using canned animation clips to move things about.

A dynamics simulation makes heavy use of the collision detection system in order to properly simulate various physical behaviors of the objects in the simulation, including bouncing off one another, sliding under friction, rolling and coming to rest. Of course, a collision detection system can be used stand-alone, without a dynamics simulation—many games do not have a "physics"

system at all. But all games that involve objects moving about in two- or three-dimensional space have some form of collision detection.

In this chapter, we'll investigate the architecture of both a typical collision detection system and a typical physics (rigid body dynamics) system. As we investigate the components of these two closely interrelated systems, we'll take a look at the mathematics and the theory that underlie them.

13.1 Do You Want Physics in Your Game?

Nowadays, most game engines have some kind of physical simulation capabilities. Some physical effects, like rag doll deaths, are simply expected by gamers. Other effects, like ropes, cloth, hair or complex physically driven machinery can add that *je ne sais quoi* that sets a game apart from its competitors. In recent years, some game studios have started experimenting with advanced physical simulations, including approximate real-time fluid mechanics effects and simulations of deformable bodies. But adding physics to a game is not without costs, and before we commit ourselves to implementing an exhaustive list of physics-driven features in our game, we should (at the very least) understand the trade-offs involved.

13.1.1 Things You Can Do with a Physics System

Here are just a few of the things you can do or have with a game physics system.

- Detect collisions between dynamic objects and static world geometry.
- Simulate free rigid bodies under the influence of gravity and other forces.
- Spring-mass systems.
- Destructible buildings and structures.
- Ray and shape casts (to determine line of sight, bullet impacts, etc.).
- Trigger volumes (determine when objects enter, leave or are inside pre-defined regions in the game world).
- Complex machines (cranes, moving platform puzzles and so on).
- Traps (such as an avalanche of boulders).
- Drivable vehicles with realistic suspensions.
- Rag doll character deaths.
- Powered rag doll: a realistic blend between traditional animation and rag doll physics.

- Dangling props (canteens, necklaces, swords), semi-realistic hair, clothing movements.
- Cloth simulations.
- Water surface simulations and buoyancy.
- Audio propagation.

And the list goes on.

We should note here that in addition to running a physics simulation at runtime in our game, we can also run a simulation as part of an offline pre-processing step in order to generate an animation clip. A number of physics plug-ins are available for animation tools like Maya. This is also the approach taken by the *Endorphin*¹ package by NaturalMotion, Inc. (<http://www.naturalmotion.com/endorphin.htm>). In this chapter, we'll restrict our discussion to runtime rigid body dynamics simulations, but offline tools are a powerful option, of which we should always remain aware as we plan our game projects.

13.1.2 Is Physics Fun?

The presence of a rigid body dynamics system in a game does not necessarily make the game fun. More often than not, the inherently chaotic behavior of a physics sim can actually detract from the gameplay experience rather than enhancing it. The fun derived from physics depends on many factors, including the quality of the simulation itself, the care with which it has been integrated with other engine systems, the selection of physics-driven gameplay elements versus elements that are controlled in a more direct manner, how the physical elements interact with the goals of the player and the abilities of the player character, and the genre of game being made.

Let's take a look at a few broad game genres and how a rigid body dynamics system might fit into each one.

13.1.2.1 Simulations (Sims)

The primary goal of a sim is to accurately reproduce a real-life experience. Examples include the *Flight Simulator*, *Gran Turismo* and *NASCAR Racing* series of games. Clearly, the realism provided by a rigid body dynamics system fits extremely well into these kinds of games.

¹NaturalMotion also offers a runtime version of Endorphin called *Euphoria*.

13.1.2.2 Physics Puzzle Games

The whole idea of a physics puzzle is to let the user play around with dynamically simulated toys. So obviously this kind of game relies almost entirely on physics for its core mechanic. Examples of this genre include *Bridge Builder*, *The Incredible Machine*, the online game *Fantastic Contraption*, and *Crayon Physics* for the iPhone.

13.1.2.3 Sandbox Games

In a sandbox game, there may be no objectives at all, or there may be a large number of optional objectives. The player's primary objective is usually to "mess around" and explore what the objects in the game world can be made to do. Examples of sandbox games include *Besiege*, *Spore*, the *LittleBigPlanet* series, and of course *Minecraft*.

Sandbox games can put a realistic dynamics simulation to good use, especially if much of the fun is derived from playing with realistic (or semi-realistic) interactions between objects in the game world. So in these contexts, physics can be fun in and of itself. However, many games trade realism for an increased fun factor (e.g., larger-than-life explosions, gravity that is stronger or weaker than normal, etc.). So the dynamics simulation may need to be tweaked in various ways to achieve the right "feel."

13.1.2.4 Goal-Based and Story-Driven Games

A goal-based game has rules and specific objectives that the player must accomplish in order to progress; in a story-driven game, telling a story is of paramount importance. Integrating a physics system into these kinds of games can be tricky. We generally give away *control* in exchange for a *realistic simulation*, and this loss of control can inhibit the player's ability to accomplish goals or the game's ability to tell the story.

For example, in a character-based platformer game, we want the player character to move in ways that are fun and easy to control but not necessarily physically realistic. In a war game, we might want a bridge to explode in a realistic way, but we also may want to ensure that the debris doesn't end up blocking the player's only path forward. In these kinds of games, physics is often not necessarily fun, and in fact it can often get in the way of fun when the player's goals are at odds with the physically simulated behaviors of the objects in the game world. Therefore, developers must be careful to apply physics judiciously and take steps to control the behavior of the simulation in various ways to ensure it doesn't get in the way of gameplay. It's usually a good idea to provide the player with a way out of difficult situations, too.

A good example of this can be found in the *Halo* series of games, where the player can press X to flip over a vehicle that has landed upside-down.

13.1.3 Impact of Physics on a Game

Adding a physics simulation to a game can have all sorts of impacts on the project and the gameplay. Here are a few examples across various game development disciplines.

13.1.3.1 Design Impacts

- *Predictability.* The inherent chaos and variability that sets a physically simulated behavior apart from an animated one is also a source of unpredictability. If something absolutely must happen a certain way every time, it's usually better to animate it than to try to coerce your dynamics simulation into producing the motion reliably.
- *Tuning and control.* The laws of physics (when modeled accurately) are fixed. In a game, we can tweak the value of gravity or the coefficient of restitution of a rigid body, which gives back some degree of control. However, the results of tweaking physics parameters are often indirect and difficult to visualize. It's much harder to tweak a force in order to get a character to move in the desired direction than it is to tweak an animation of a character walking.
- *Emergent behaviors.* Sometimes physics introduces unexpected features into a game—for example, the rocket-launcher jump trick in *Team Fortress Classic*, the high-flying exploding Warthog in *Halo* and the flying “surfboards” in *PsyOps*.

In general, the game design should usually drive the physics requirements of a game engine—not the other way around.

13.1.3.2 Engineering Impacts

- *Tools pipeline.* A good collision/physics pipeline takes time to build and maintain.
- *User interface.* How does the player control the physics objects in the world? Does he or she shoot them? Walk into them? Pick them up? Does he or she hold them using virtual arms, as in *Trespasser*? Or using a “gravity gun,” as in *Half-Life 2*?
- *Collision detection.* Collision models intended for use within a dynamics simulation may need to be more detailed and more carefully constructed than their non-physics-driven counterparts.

- *AI.* Pathing may not be predictable in the presence of physically simulated objects. The engine may need to handle dynamic cover points that can move or blow up. Can the AI use the physics to its advantage?
- *Misbehaved objects.* Animation-driven objects can clip slightly through one another with few or no ill effects. But when driven by a dynamics simulation, objects may bounce off one another in unexpected ways or jitter badly. Collision filtering may need to be applied to permit objects to interpenetrate slightly. Mechanisms may need to be put in place to ensure that objects settle and go to sleep properly.
- *Rag doll physics.* Rag dolls require a lot of fine-tuning and often suffer from instability in the simulation. An animation may drive parts of a character's body into penetration with other collision volumes—when the character turns into a rag doll, these interpenetrations can cause enormous instability. Steps must be taken to avoid this.
- *Graphics.* Physics-driven motion can have an effect on renderable objects' bounding volumes (where they would otherwise be static or more predictable). The presence of destructible buildings and objects can invalidate some kinds of precomputed lighting and shadow methods.
- *Networking and multiplayer.* Physics effects that do not affect gameplay may be simulated exclusively (and independently) on each client machine. However, physics that has an effect on gameplay (such as the trajectory that a grenade follows) must be simulated on the server and accurately replicated on all clients.
- *Record and playback.* The ability to record gameplay and play it back at a later time is very useful as a debugging/testing aid, and it can also serve as a fun game feature. This feature is difficult to implement because it requires every engine system to behave in a deterministic manner, so that everything will play out exactly in the same way during playback as it did when the recording was made. If your physics simulation isn't deterministic, this can become a major fly in the ointment.

13.1.3.3 Art Impacts

- *Additional tool and workflow complexity.* The need to rig up objects with mass, friction, constraints and other attributes for consumption by the dynamics simulation makes the art department's job more difficult as well.
- *More complex content.* We may need multiple visually identical versions of an object with different collision and dynamics configurations for dif-

ferent purposes—for example, a pristine version and a destructible version.

- *Loss of control.* The unpredictability of physics-driven objects can make it difficult to control the artistic composition of a scene.

13.1.3.4 Other Impacts

- *Interdisciplinary impacts.* The introduction of a dynamics simulation into your game requires close cooperation between engineering, art, audio and design.
- *Production impacts.* Physics can add to a project's development costs, technical and organizational complexity and risk.

Having explored the impacts, most teams today do choose to integrate a rigid body dynamics system into their games. With some careful planning and wise choices along the way, adding physics to your game can be rewarding and fruitful. And as we'll see below, third-party middleware is making physics more accessible than ever.

13.2 Collision/Physics Middleware

Writing a collision system and rigid body dynamics simulation is challenging and time-consuming work. The collision/physics system of a game engine can account for a significant percentage of the source code in a typical game engine. That's a lot of code to write and maintain!

Thankfully, a number of robust, high-quality collision/physics engines are now available, either as commercial products or in open source form. Some of these are listed below. For a discussion of the pros and cons of various physics SDKs, check out the online game development forums (e.g., http://www.gamedev.net/community/forums/topic.asp?topic_id=463024).

13.2.1 ODE

ODE stands for “Open Dynamics Engine” (<http://www.ode.org>). As its name implies, ODE is an open source collision and rigid body dynamics SDK. Its feature set is similar to a commercial product like Havok. Its benefits include being free (a big plus for small game studios and school projects!) and the availability of full source code (which makes debugging much easier and opens up the possibility of modifying the physics engine to meet the specific needs of a particular game).

13.2.2 Bullet

Bullet is an open source collision detection and physics library used by both the game and film industries. Its collision engine is integrated with its dynamics simulation, but hooks are provided so that the collision system can be used stand-alone or integrated with other physics engines. It supports *continuous collision detection* (CCD)—also known as *time of impact* (TOI) collision detection—which, as we’ll see below, can be extremely helpful when a simulation includes small, fast-moving objects. The Bullet SDK is available for download at <http://code.google.com/p/bullet/>, and the Bullet wiki is located at http://www.bulletphysics.com/mediawiki-1.5.8/index.php?title=Main_Page.

13.2.3 TrueAxis

TrueAxis is another collision/physics SDK. It is free for non-commercial use. You can learn more about TrueAxis at <http://trueaxis.com>.

13.2.4 PhysX

PhysX started out as a library called Novodex, produced and distributed by Ageia as part of their strategy to market their dedicated physics coprocessor. It was bought by NVIDIA and retooled so that it can run using NVIDIA’s GPUs as a coprocessor. (It can also run entirely on a CPU, without GPU support.) It is available at http://www.nvidia.com/object/nvidia_physx.html. Part of Ageia’s and NVIDIA’s marketing strategy has been to provide the CPU version of the SDK entirely for free, in order to drive the physics coprocessor market forward. Developers can also pay a fee to obtain full source code and the ability to customize the library as needed. PhysX is now combined with APEX, NVIDIA’s scalable multiplatform dynamics framework. PhysX/APEX is available for Windows, Linux, Mac, Android, Xbox 360, PlayStation 3, Xbox One, PlayStation 4 and Wii.

13.2.5 Havok

Havok is the gold standard in commercial physics SDKs, providing one of the richest feature sets available and boasting excellent performance characteristics on all supported platforms. (It’s also the most expensive solution.) Havok is comprised of a core collision/physics engine, plus a number of optional add-on products including a vehicle physics system, a system for modeling destructible environments and a fully featured animation SDK with direct integration into Havok’s rag doll physics system. It is available on Xbox 360,

PlayStation 3, Xbox One, PlayStation 4, PlayStation Vita, Wii, Wii U, Windows 8, Android, Apple Mac and iOS. You can learn more about Havok at <http://www.havok.com>.

13.2.6 Physics Abstraction Layer (PAL)

The Physics Abstraction Layer (PAL) is an open source library that allows developers to work with more than one physics SDK on a single project. It provides hooks for PhysX (Novodex), Newton, ODE, OpenTissue, Tokamak, TrueAxis and a few other SDKs. You can read more about PAL at <http://www.adrianboeing.com/pal/index.html>.

13.2.7 Digital Molecular Matter (DMM)

Pixelux Entertainment S.A., located in Geneva, Switzerland, has produced a unique physics engine that uses finite element methods to simulate the dynamics of deformable bodies and breakable objects, called Digital Molecular Matter (DMM). The engine has both an offline and a runtime component. It was released in 2008 and can be seen in action in LucasArts' *Star Wars: The Force Unleashed*. A discussion of deformable body mechanics is beyond our scope here, but you can read more about DMM at <http://www.pixeluxentertainment.com>.

13.3 The Collision Detection System

The primary purpose of a game engine's collision detection system is to determine whether any of the objects in the game world have come into *contact*. To answer this question, each logical object is represented by one or more geometric *shapes*. These shapes are usually quite simple, such as spheres, boxes and capsules. However, more complex shapes can also be used. The collision system determines whether or not any of the shapes are *intersecting* (i.e., overlapping) at any given moment in time. So a collision detection system is essentially a glorified geometric intersection tester.

Of course, the collision system does more than answer yes/no questions about shape intersection. It also provides relevant information about the nature of each contact. Contact information can be used to prevent unrealistic visual anomalies on-screen, such as objects *interpenetrating* one another. This is generally accomplished by moving all interpenetrating objects apart prior to rendering the next frame. Collisions can provide *support* for an object—one or more contacts that together allow the object to come to rest, in equilibrium with gravity and/or any other forces acting on it. Collisions can also be used